DEALFLOW360

Solution Report

Four people. One day. One connected sales workflow.

A revised product and architecture proposal using the team's Spring Boot experience to connect quotation, negotiation, approval, fulfillment, and billing.

Core decision	Spring Boot + JPA / PostgreSQL, with a Next.js frontend.
Live updates	Authenticated WebSockets; committed outbox; RabbitMQ optional later.
Edge-case review	All 23 submitted cases resolved with explicit policies and tests.
Machine learning	No trained model required; Deal Lab remains the creative extension.

Proposed design and implementation specification. Application code and deployment are future work; required features and optional innovations are distinguished throughout.

Contents

Revision 2 • 5 September 2026 • Team: 4 • Build window: 24 hours

Update: The team confirmed practical Spring Boot experience. This edition replaces the TypeScript backend proposal with Spring Boot and incorporates all 23 submitted edge cases. The previous edition is preserved in `_archive/revision-1`.

Product promise: A sales workspace that explains the consequences of a deal, governs its approval, and carries the accepted terms through stock allocation and billing.

Status: Proposed solution, not an implemented or benchmarked application. The companion Implementation Plan contains the build sequence, contracts, formulas, acceptance tests, and demo script.

1. Executive decision

Use **Spring Boot with Java 21 for the backend**, Spring Data JPA/Hibernate with PostgreSQL for persistence, Spring Security for API authorization, and authenticated STOMP over WebSocket for live updates. Retain Next.js/React/TypeScript for the frontend. Keep Supabase as the managed PostgreSQL and identity provider; Spring verifies its access tokens and enforces application roles and record ownership.

The team confirmed practical Spring Boot experience. That changes the earlier tradeoff: Spring's transaction, persistence, security, and messaging ecosystem is now a useful productivity advantage rather than a framework to learn during the event. This is a team-specific recommendation, not a claim that JavaScript backends lack customization, WebSockets, queues, or transaction support.

Build **one modular Spring backend**, one frontend, and one database. Keep quote finalization, reservations, and initial billing in a database transaction. Microservices would turn that operation into a distributed consistency problem. RabbitMQ is an optional asynchronous-processing stage after the core passes, not a required route through which all HTTP requests must travel.

Use a database outbox for committed notification events. The initial dispatcher pushes authorized UI invalidations through Spring's simple WebSocket broker. A later RabbitMQ dispatcher/worker can reuse those events. Neither a WebSocket frame nor a broker acknowledgment is the source of truth for an approved quote or paid invoice.

No trained machine-learning model is required. Purchase-history statistics support recommendations; explicit policies support approvals, allocation, proration, and alerts. The optional creative extension remains **Deal Lab**, which compares valid quotation alternatives using these same engines.

Four people have at most 96 gross person-hours, before coordination and rehearsal. The full brief is ambitious. Preserve required business behavior, deploy both applications early, and cut optional services/innovation before sacrificing correctness.

2. What the problem is asking for

The brief describes a connected B2B quotation-to-cash system, with seven responsibilities:

Responsibility	Required result	Why it matters
Discount governance	Configurable customer/category ceilings; automatic Manager and Finance routing; audit history	Prevent unauthorized concessions
Recommendations	Ranked product suggestions, promotions, and immediate margin impact	Improve the deal while it is being built
Fulfillment	Stock-aware warehouse split, override, backorder, and consolidation after receipt	Make delivery decisions reflect inventory

Responsibility	Required result	Why it matters
Hybrid billing	One-time items and recurring plans, schedules, proration, cancellation, credits	Preserve the commercial agreement through billing
Customer negotiation	Separate restricted portal, line comments, change requests, counter-discounts, confirmation	Collaborate on the actual quotation
Deal monitoring	Stalled quotes, unusual discounts, promise slippage, nudges/escalations	Surface problems early
Configuration/reporting	Working setup forms, filters, PDF and spreadsheet exports	Make behavior configurable and results inspectable

The brief explicitly permits any language, framework, or database. Building an Odoo module is not required by this PDF. It explicitly rejects fake approval, inventory, and billing logic. The customer portal must enforce restricted access on the server. [Brief, pp. 2–10]

The optional wording applies to A6, the recommendation rule-configuration screen. The actual recommendation panel is required by B5 and the quick test. Multi-company and multi-currency execution are bonuses. [Brief, pp. 5, 7, 10–11]

3. People and their complete journeys

Sales representative

Sign up/log in → select customer → build a quotation → see live totals, margin, risk reasons, and suggestions → add/dismiss a suggestion → submit for required approval → share the customer portal link → respond to negotiation → follow fulfillment and billing.

Manager and Finance/Operations

Manager reviews the exact submitted quote version and the policy reasons. They approve, reject, or return it for revision. Finance acts after Manager when required. Operations reviews warehouse allocation and backorders; Finance records payments and handles credits/refund records. Actions are attributed and time-stamped.

Customer

Log into the portal → see only their quotations → comment on a particular line or propose revised commercial terms → see whether approval is pending → accept the exact terms displayed. An acceptance with outstanding approvals is conditional and cannot release stock or create a billable order.

Administrator

Configure products/variants, prices, customer tiers, category ceilings, approval thresholds, warehouses, stock, replenishment thresholds, shipping weights, plans, proration/cancellation rules, and alert thresholds. Configuration changes affect future evaluations; submitted versions preserve their applicable policy snapshot.

4. The product experience

Use a desktop-first workspace that fits a laptop projector. A quotation has one detail page with tabs for **Build, Approvals, Customer Activity, Fulfillment, Billing, and History**. Avoid forcing users through unrelated pages to understand one deal.

The builder has three regions: searchable catalog, editable quote lines, and a context panel. The context panel shows required approval with reasons, recommendation cards, and fulfillment availability. Display the amount due initially separately from recurring commitments. Label recurring margin calculations with their billing period.

The portal uses its own layout and API responses. It displays customer prices and terms, without internal costs, margins, risk thresholds, other customers, or unrelated inventory detail. Provide a clear version label and pending-approval banner.

The manager dashboard prioritizes actionable deals. Each alert opens the relevant quotation and a real action. Use consistent colors with written labels: green for ready, amber for waiting/risk, red for blocked. Charts must link to live filtered records; decorative metrics add little value.

5. Architecture and framework decisions

5.1 Spring Boot versus the alternatives

Choice	Fit for this team	Important tradeoff
Spring Boot modular backend	Recommended: existing practical experience, strong fit for transactional workflows	Two runtimes/deployments and explicit frontend/backend contracts
Next.js full-stack backend	Still viable for a TypeScript-first team and a short event	Persistent WebSockets normally need a separate suitable runtime when deployed serverlessly
NestJS backend	Viable structured TypeScript alternative with WebSocket/queue integrations	Switching to an unfamiliar framework has no demonstrated benefit here
Spring microservices from day one	Defer	Quotes, stock, and invoices would need distributed recovery, messaging, and additional deployments

JPA reduces mapping and repository boilerplate. It does not eliminate connection limits, deadlocks, stale data, or overselling. Use HikariCP, explicit service transactions, optimistic versions, pessimistic inventory locks, database constraints, and Flyway migrations. Spring Data exposes locking and transaction mechanisms; the team must apply them correctly. Spring Data JPA locking, transactionality

RabbitMQ buffers asynchronous jobs and controls consumer workload. It is not an HTTP load balancer or an automatic database scaling solution. The baseline manages traffic through bounded requests, pagination, debouncing, connection-pool limits, and measured indexes. A queue is justified later for notifications, exports, and external integrations that can complete asynchronously.

5.2 Selected stack

Layer	Selection	Purpose
Frontend	Next.js 16, React, TypeScript, Tailwind, shadcn/ui	Internal workspace and genuinely separate portal
Client data/forms	TanStack Query, React Hook Form, Zod	Validated input and server-authoritative state
Backend	Java 21, patched Spring Boot 4.1.x, Spring MVC	REST controllers and modular domain services
Identity/security	Supabase Auth + Spring Security OAuth2 Resource Server	JWT verification plus database-backed role/ownership checks
Persistence	PostgreSQL, Spring Data JPA/Hibernate, HikariCP, Flyway	Mapping, transactions, connection pooling, schema migration
Money/calendar	Java BigDecimal, integer persisted minor units, java.time	Exact rounding and explicit calendar periods
Live updates	Spring WebSocket/STOMP + `@stomp/stompjs`	Authenticated per-user updates after commit
Jobs/events	Spring scheduler, DB leases, transactional outbox	Browser-independent billing and recoverable notifications
Exports	Apache PDFBox and Apache POI	Server-authorized PDF, XLSX, and legacy XLS if needed
Verification	Spring Boot Test/JUnit, Testcontainers PostgreSQL, Playwright	Rules, real DB concurrency, auth and browser journeys
Hosting	Vercel frontend, one persistent Spring container on Railway, Supabase DB/Auth	Long-running scheduler/WebSocket support
Optional later	Spring AMQP + RabbitMQ	Durable asynchronous processing beyond the baseline

Choose compatible patched releases through Spring Initializr and Boot's dependency management; do not independently combine arbitrary Hibernate/Security versions. Java 21 is compatible with the checked Boot requirements. Keep an already-tested supported Spring baseline if a major-version migration would otherwise consume the event. Spring Boot requirements

WebSocket subscriptions are private and authenticated. Reconnect triggers an authorized REST refresh, because push messages can be missed. Spring's simple broker is sufficient for the chosen single-instance deployment and is not a clustered broker. Spring broker documentation

Deploy the Spring server on a host that supports persistent processes and WebSocket upgrades. Railway documents this support; verify account resources and keep the instance running rather than assuming a sleeping/free service can run billing jobs. Railway networking

6. Required intelligence versus creative extensions

Capability	Already in the brief?	Proposed implementation
Upsell/cross-sell	Yes	Purchase-history association counts plus promotion and margin rules
Discount approval	Yes	Explainable, configurable risk calculation and sequential approvals
Warehouse optimization	Yes	Deterministic search over a small warehouse set
Anomaly/stalled alerts	Yes	Statistical comparisons and explicit thresholds
Customer counteroffer	Yes	Versioned proposal with automatic re-evaluation
Deal Lab alternatives	No	Counterfactual alternatives evaluated by the same real engines
Unified decision explanation	Extends required audit/risk views	Connect pricing, approval, inventory, and billing reasons in one readable view

Do not describe baseline features as extra innovation. Do not claim worldwide novelty without evidence. The distinctive contribution is the integration, clarity, and quality of execution.

Signature extension: Deal Lab

For a quote awaiting approval, generate at most three candidate alternatives from the actual catalog and policy:

1. **Approval-ready:** reduce excessive discounts to configured safe values, and recheck both aggregate policy and margin constraints.
2. **Margin-focused:** add or substitute an eligible product only when the resulting customer price and margin meet stated constraints.
3. **Delivery-focused:** use an explicitly configured equivalent product with available stock, displaying any price or specification change.

Each candidate shows the change from the current quote: price, margin amount/percentage, approval chain, shipment estimate, and unfilled quantity. Report when no feasible alternative exists. A rep applies one explicitly; the server revalidates current stock, prices, policy, and quote version before saving a new revision. A scenario never reserves inventory or alters an approved deal by itself.

Scope control: implement approval-ready first. The other two are stretch features. Label any heuristic result as a suggested alternative, not a provably optimal deal.

Lower-cost extension: one decision explanation

Example: “Finance is needed because the service discount exceeds its category ceiling by 8 percentage points. One laptop is backordered. This order includes INR 39,300 once and INR 3,000 per month before tax.”

Every sentence comes from structured engine outputs. Links open the triggering line, approval step, stock shortage, or billing schedule. It requires no language model and gives judges a clear explanation of the system's reasoning.

7. Do we need to build a machine-learning model?

No. No requirement in this PDF forces model training.

Problem	Suitable 24-hour technique	Why training is unnecessary
Product recommendations	Co-purchase confidence with promotion boost and margin filter	Uses actual order history directly
Discount anomaly	Historical mean/standard deviation plus absolute difference threshold	Transparent, testable, works with small demo history
Approval	Policy formula and state machine	This is a business constraint, not a prediction
Warehouse allocation	Small combinatorial search	This is an optimization problem
Proration	Calendar-based arithmetic	Billing needs deterministic amounts
Deal Lab	Generate a few candidates and evaluate them	Reuses the existing engines

Use representative synthetic history, clearly labeled as demo data. It must contain real stored order lines that the recommendation query aggregates; do not return fixed recommendation cards. When history is insufficient, return eligible promoted/category alternatives with a “limited history” reason, or no suggestion.

An optional LLM could paraphrase a structured deal explanation or draft a negotiation response for a human to review. It must not decide prices, approve deals, reserve stock, issue invoices, or send messages. It is not needed for the planned demo and is excluded from the 24-hour critical path.

After the hackathon, a learned recommendation/ranking model could be evaluated on time-separated real interaction data. Compare it with the rules baseline using ranking quality and business outcomes; do not claim accuracy or conversion lift from synthetic data. Predictive win probability needs reliable outcome labels, sufficient history, and calibration before it is useful.

8. Decisions the brief leaves open

Ambiguity	Proposed explicit decision
Exact blended-risk formula	Evaluate worst line, value-weighted excess, total excess value, and overall discount/margin limits; route to the highest level
Approval versus customer confirmation order	Both are independent gates; create an executable order only when the current version meets both
Stock before customer acceptance	Preview only; reserve after all gates pass; accepted backorder policy determines shortage behavior
Meaning of hybrid total	Show one-time amount, each recurring plan/cadence, and initial amount due separately
Proration convention	Calendar-day fraction with start-inclusive/end-exclusive periods
Portal counteroffer	Create a candidate revision, immediately route its risk, require seller adoption and customer acceptance before execution
Real-time behavior	Authenticated WebSocket invalidation after commit, REST refetch, and polling fallback
Payment scope	Persist payment/credit/refund records; a real payment gateway is a future integration
Multi-currency	Currency-aware price records; execute the hackathon in INR with no FX conversion
“XLS” export	Apache POI supports real XLS and XLSX; expose the chosen workbook format clearly

These are team design choices, not formulas supplied by the organizers. Record them in the README and make the behavior visible in the demo. The numerical defaults are illustrative commercial policies, not legal or financial rules.

9. What success looks like

The required deliverable is a working application, seeded data, a five-minute live demo with two complete journeys, a one-page architecture/data-model diagram, and a short roadmap. [Brief, p. 10]

The demo should prove:

- A normal quote completes without unnecessary approval, accepts a recommendation, reserves/dispatches stock, creates an invoice, and becomes paid after a recorded receipt.
- A mixed hardware/service/subscription quote triggers sequential approval, receives a customer counteroffer, invalidates earlier approvals, clears the new gates, splits across warehouses, and handles a real backorder receipt.
- Billing separates one-time and recurring charges; a configured mid-cycle quantity change creates a calculated adjustment; cancellation follows the configured credit policy.
- Another customer's quotation is inaccessible, duplicate actions do not duplicate financial records, and stock does not become negative.
- Reporting and alerts reflect persisted activity, not hardcoded dashboards.

The five-minute script cannot show every edge case. Automated acceptance evidence covers the rest. Performance goals are targets, not measured claims: typical quote evaluation below 500 ms on the demo dataset; user-visible freshness within five seconds; zero duplicate orders/invoices in retry tests.

10. Delivery strategy and risks

Members own delivery modules: A owns platform/quote/governance; B owns fulfillment and its UI; C owns recurring billing/payments; D owns portal/recommendations/reporting. Assign Java changes to members with that experience and pair frontend-focused members through explicit OpenAPI contracts. Merge small working changes every two hours. Agree on keys, money units, statuses, REST schemas, and Java service boundaries at the start.

The first integrated ordinary quote should work by hour 8. Both ordinary and negotiated mixed flows should work by hour 16. Required edge cases, exports, and alerts should pass by hour 20. Innovation gets a strictly bounded slot after that; the final hours are for fixes and rehearsal.

Risk	Containment
Four developers create incompatible modules	OpenAPI contract, generated frontend types, and centrally reviewed Flyway migrations in the first hour
Quote edits accidentally retain approval	Immutable submitted versions; version-specific decisions and acceptances
Concurrent confirmations oversell stock	Row locks and atomic reservation with nonnegative constraints
Recurring jobs create duplicate invoices	Unique charge keys, job locks, and retry-safe writes
Portal reveals internal data	Separate DTOs, ownership checks, and negative access tests
Configuration is only cosmetic	Tests change a setting and verify the next evaluation changes
Deployment fails near the deadline	Deploy frontend, Spring API and one database query by hour 2; test the actual WebSocket endpoint early

Risk	Containment
Innovation consumes completion time	Cut Deal Lab variants/LLM before cutting baseline requirements

11. Architecture decision record

Context: Four developers, one day, confirmed Spring Boot experience, many related transactions, and 23 submitted edge cases.

Decision: Next.js frontend, a modular Spring Boot backend, PostgreSQL/JPA with explicit locking and migrations, managed identity, authenticated WebSockets, and a transactional outbox. RabbitMQ and microservice extraction are optional later steps.

Consequences: The stack uses familiar backend tools but requires two deployment pipelines and cross-language API contracts. Persistent scheduling and WebSockets are straightforward on the chosen backend runtime. Locks, versions, and idempotency remain essential regardless of JPA.

Evolution triggers: Add RabbitMQ when measurable async backlog/external integration warrants it. Add a distributed WebSocket broker/routing arrangement before horizontal replicas. Extract workers before splitting tightly coupled quote/order/stock/billing transactions. Consider separate services only with independent scaling/ownership requirements and explicit compensation/idempotency design.

11.1 Edge-case decisions incorporated in revision 2

All 23 submitted cases are resolved and paired with tests in section 19 of the Implementation Plan. The important changes are:

- Apply order discounts consistently and test each line's combined discount; below-limit lines never cancel another line's positive violation. Promotion is not a policy exemption.
- Block zero-revenue/100%-discount lines in this build. Non-positive contribution at a positive price requires Finance. The margin percentage at zero revenue is undefined, not zero.
- Serialize version-specific approval actions, invalidate pending work after edits/cancellation, and provide audited role-qualified reassignment when an approver is unavailable.
- Re-evaluate all changed terms even when a requested discount decreases; bypass approval only when the new version meets the current policy.
- Permit a pure backorder only under explicit accepted backorder terms. An invalid manual warehouse allocation is rejected rather than silently changed.
- Use a cost-aware shipping score with a per-extra-shipment penalty, so two inexpensive local shipments can beat one expensive remote shipment.
- Prorate the **additional ten units** in a 10-to-20 upgrade using actual dates. “Day 15” and “15 elapsed days” are different inputs.
- Keep accepted hardware discounts intact on support cancellation; no undisclosed clawback is added. Backdated activation is rejected in this hackathon scope.
- Require idempotent counter submissions, reject invalid discounts, require exact-version acceptance, and make post-order negotiation read-only.
- Separate activity from external/business progress. Repeated rep edits do not reset the no-progress timer. New reps do not get an invented zero-percent statistical baseline.

These are explicit proposed policies. The edge-case attachment raises questions; it does not itself define the correct policy or authorize external transactions.

12. Sources and interpretation

Primary requirements: **DealFlow360.pdf**, 13 pages, provided by the team at `C:/Users/Nikhil1616/Downloads/DealFlow360.pdf`. References to page numbers above refer to that PDF. The linked Excalidraw mockup was not retrievable, so no unobserved mockup behavior is assumed.

The PDF is source material for the product requirements, not an instruction to execute its login, payment, or external-message walkthrough now. This deliverable is a report and implementation plan only.

Revision 2 sources: the team's supplied `pasted-text.txt` containing 23 edge cases, and official Spring/JPA/WebSocket/RabbitMQ/Apache references linked in this edition and the plan.

Technical references checked on 5 September 2026: Next.js, Supabase SSR, PostgreSQL connections, Spring transactions, PostgreSQL locking, Spring scheduling, shadcn/ui, Apache PDFBox. Stack selection, risk formulas, UX, effort allocation, and innovation concepts are this report's proposed design judgments.